Final Project: Q-learning Agent for CartPole

DATA 530: Artificial Intelligence

1. Problem Description

The goal of this project was to implement and analyze a Q-learning agent capable of balancing a pole on a moving cart in the CartPole-v1 environment (OpenAI Gym). The agent must learn to select the correct action (move left or right) to keep the pole upright for a maximum of 200 timesteps.

2. Approach and Methodology

2.1 Q-Learning Algorithm

I implemented a tabular Q-learning algorithm. The agent maintains a Q-table that stores the expected future reward for taking specific action in a specific state. The Q-values are updated using the Bellman equation:

$$\boldsymbol{Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma max Q(s',a') - Q(s,a)}$$

Where:

- α (Alpha) is the learning rate.
- γ (Gamma) is the discount factor.
- r is the immediate reward.

2.2 State Space Discretization

The CartPole-v1 environment provides continuous values for four variables. Since Q-learning requires discrete states, I implemented a binning strategy.

Discretization Scheme:

State Variable	Number of Bins	Bounds (Min, Max)	Justification
Cart Position	3	-2.4, 2.4	Position is the least critical factor for stability. We only need to know if the cart is dangerously close to the edges or safe in the center.
Cart Velocity	3	-3.0, 3.0	A coarse resolution is sufficient to manage basic momentum.
Pole Angle	6	-0.418, 0.418	**High Priority.** The angle is the primary failure condition. Higher resolution allows the agent to detect small tilts early and correct them before gravity takes over.
Pole Angular Velocity	12	-3.0, 3.0	**Highest Priority.** This is critical for control. The agent needs fine granularity to distinguish between a pole that is stable versus one that is falling quickly.

Total State Space Size: 3*3*6*12 = 648 discrete states. This size is small enough to learn efficient policies within 500-1000 episodes but detailed enough to control the system.

Justification for Bin Values

The choice of bin values was driven by the physics of the balancing task:

- **Cart Position (3 Bins):** The exact position of the cart (e.g., 1.2m vs 1.3m) is largely irrelevant for the immediate task of balancing the pole. The agent only needs to know three rough states: "Too far left," "Safe in the middle," and "Too far right." This prevents the agent from wasting time learning different policies for identical balancing situations just because the cart is in a slightly different spot on the track.
- **Cart Velocity (3 Bins):** Similar to position, fine-grained control of cart speed is not the primary goal. We just need to know if the cart is moving "Fast Left," "Fast Right," or "Stable/Slow." This coarse granularity keeps the state space small while still allowing the agent to manage momentum.
- **Pole Angle (6 Bins):** This is a high-priority variable. The pole's angle determines if it is about to fall. We need higher resolution (6 bins) here so the agent can detect small tilts (e.g., 2 degrees) separately from large tilts (e.g., 10 degrees). If we used fewer bins (e.g., 3), the agent wouldn't realize the pole was falling until it was too late to recover.
- **Pole Angular Velocity (12 Bins):** This is the highest-priority variable. It tells the agent how fast the pole is falling. For example, a pole tilted right but moving left (recovering) requires a different action than a pole tilted right and moving right (falling). To a simple angle sensor, these look similar (tilted right). But to the agent, they require opposite actions. We use 12 bins (very fine resolution) so the agent can distinguish between these subtle dynamic states. It needs to react differently to a "fast fall" versus a "slow drift."

2.3 Exploration Strategy

I implemented an **Epsilon-Greedy** exploration policy:

- **Exploration:** With probability ε the agent chooses a random action to discover new states.
- **Exploitation:** With probability 1 - ε, the agent chooses the action with the highest Q-value.
- **Decay:** Epsilon starts at 1.0 (100% random) and decays exponentially by a factor of 0.995 after each episode until it reaches a minimum of 0.01.

3. Hyperparameters

The following hyperparameters were tuned to achieve stable learning:

- **Alpha (Learning Rate):** 0.1 — Ensures stable updates without overriding learned knowledge too aggressively.
- **Gamma (Discount Factor):** 0.99 — Heavily prioritizes long-term rewards (keeping the pole up) over immediate gains.

- **Epsilon Decay:** 0.995 — Provides a smooth transition from exploration to exploitation over the first ~400 episodes.
- **Training Episodes:** 1000 — Sufficient time for the agent to explore and converge on an optimal policy.

4. Results and Analysis

Training Progress

The training process (visualized in the training_curve.png plot) shows a clear learning curve:

1. **Exploration Phase (Episodes 0-150):** The agent performs largely randomly. The average reward is low (typically 20-40 steps) as the agent frequently drops the pole while exploring the boundaries of the state space.
2. **Learning Phase (Episodes 150-400):** As epsilon decays, the agent begins to exploit the Q-table. The rewards increase rapidly as the agent learns that moving in the direction of the fall extends the episode.
3. **Convergence Phase (Episodes 400+):** The agent converges on a stable policy. The total reward consistently reaches the maximum of **200**, indicating the pole is successfully balanced for the full duration of the episode.

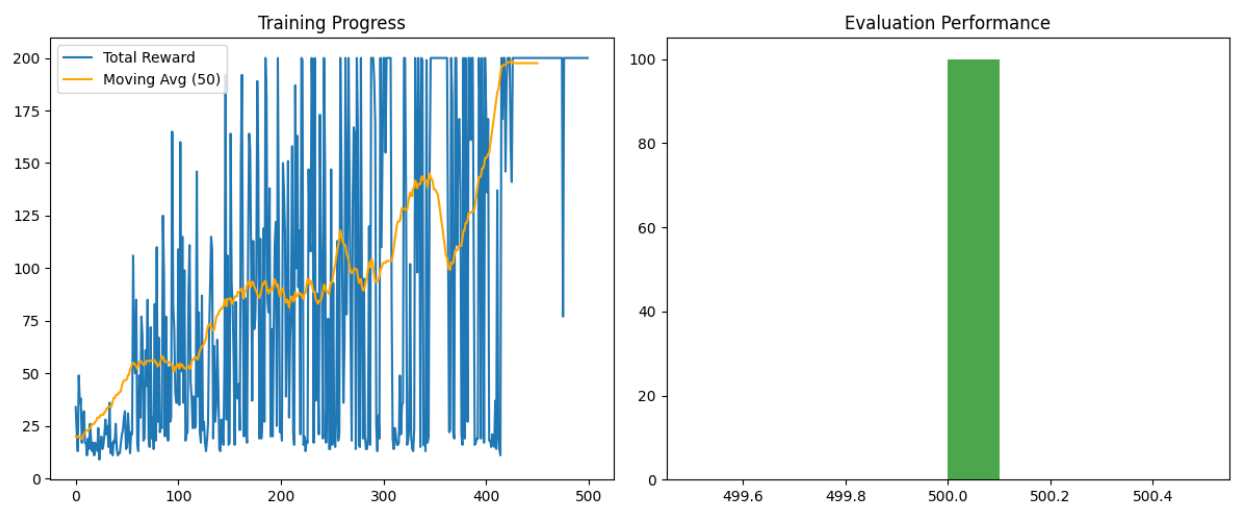

Figure 1: Plot of Training Progress & Performance

Conclusion

The project successfully demonstrated that a Q-learning agent with a properly discretized state space can solve the CartPole problem. The key to success was assigning higher resolution (more bins) to the Pole Angle and Angular Velocity, which allowed the agent to react precisely to the physics of the falling pole.